

Projetando, codificando e simulando para MCU

Assembly do ATmega328

Prof. Roberto de Matos

roberto.matos@ifsc.edu.br



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
São José

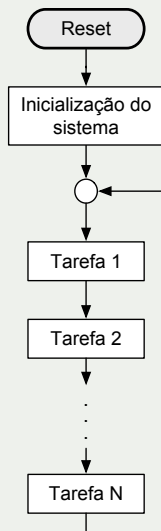
Objetivo

- Entender como projetar um *software* embarcado básico
- Entender como codificar um *software* em Assembly
- Introdução à simulação

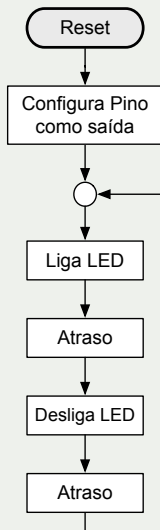
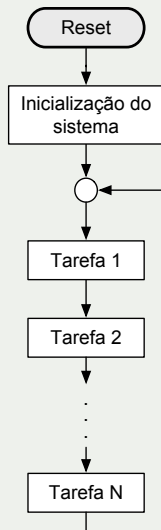


Software embarcado básico

- Conhecido como *Round-robin* ou “loopão”
- Simplicidade
- Funcionamento básico:
 - Configura o sistema logo depois do *reset*
 - Executa as tarefas de forma sequencial e repetida
- Exemplo: Pisca-pisca



- Conhecido como *Round-robin* ou “loopão”
- Simplicidade
- Funcionamento básico:
 - Configura o sistema logo depois do *reset*
 - Executa as tarefas de forma sequencial e repetida
- Exemplo: Pisca-pisca



Fluxograma

Fluxograma - Símbolos básicos

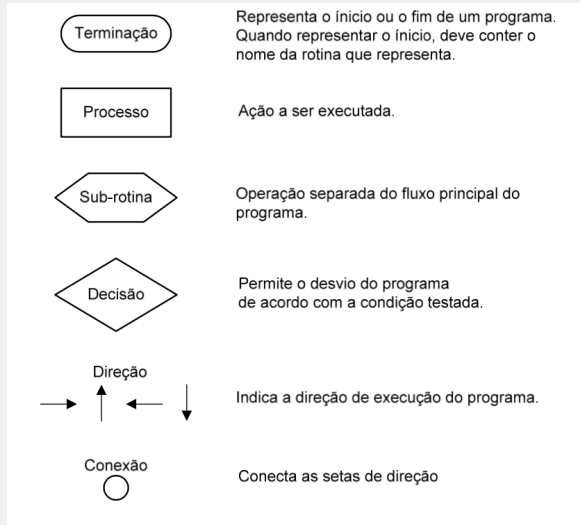
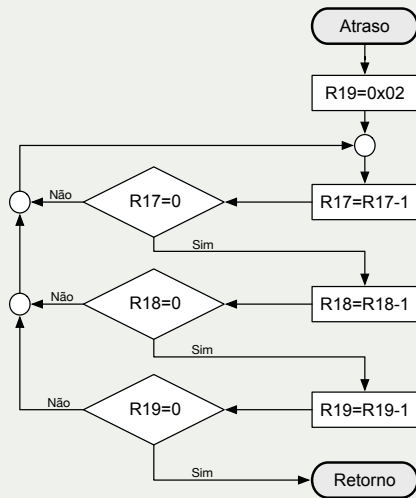
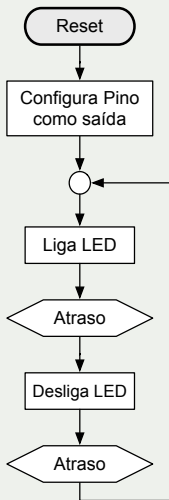


Fig. 4.1– Símbolos básicos para o desenho de fluxogramas.



Fluxograma - Exemplo



Programando em *Assembly*

- Planejar e modelar o código
- Conhecer a arquitetura (hierarquia de memória, periféricos, etc.)
- Conhecer as instruções *assembly*¹
- Conhecer as diretivas do montador² (*assembler*)

¹Manual do conjunto de instruções do AVR

²Manual online do montador



■ Sintaxe do arquivo:

```
1 [Rótulo:] .diretiva [operandos] [comentários]
2 [Rótulo:] instrução [operandos] [comentários]
3 Comentários
4 Linhas em branco
```

- Entre [] é opcional
- Diretivas → Montador
- Instruções → CPU
- Todo texto após “;” ou “//” é comentário



■ Sintaxe do arquivo:

```
1 [Rótulo:] .diretiva [operandos] [comentários]
2 [Rótulo:] instrução [operandos] [comentários]
3 Comentários
4 Linhas em branco
```

■ Entre [] é opcional

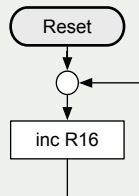
■ Diretivas → Montador

■ Instruções → CPU

■ Todo texto após “;” ou “//” é comentário

■ Exemplo:

```
1 label: .DEF AUX=R16 ; nomeia o registrador R16 como AUX (Diretiva)
2
3 start: INC r16      ; incrementa o r16 (Instrução)
4        RJMP start  ; salta para o rótulo "start" (Instrução)
5                ; linha com comentário
6                // outro comentário
```



- Definição do seguimento de dados ou código: **.DSEG, .CSEG, .ESEG**
- Reserva de bytes na memória SRAM (variável): **.BYTE**
- Reserva de bytes/words (constantes): **.DB, .DW**
- Definir um nome simbólico para um registrador: **.DEF**
- Setar um símbolo a partir de uma expressão: **.EQU, .SET**
- Setar um endereço de origem: **.ORG**
- Incluir um arquivo fonte: **.INCLUDE**

³Verificar [lista completa de diretivas](#) no manual online do montador.



- Decimal: 10, 255
- Hexadecimal: 0x0a, \$0a, 0xff, \$ff
- Binário: 0b00110101, 0b1100_1010
- Octal (zero na frente): 010, 077



Ambiente de desenvolvimento e simulação

Abrindo o Projeto

- Descompactar o arquivo AOC129004.X.zip em um local conhecido.
- Abra o MPLAB X IDE
- Clique em *File -> Open Project*
- Selecione o diretório descompactado
- Clique em *Open Project*
- No painel no canto superior esquerdo *Projects*, expanda o projeto e o diretório *Source Files*
- Abra o arquivo `aula1.asm`
- Organize as janelas para a simulação conforme sugestão



Sugestão de organização

MPLAB X IDE v5.45 - mic29004 : programming

3024/509.1MB PC: 0x0 ithsvnzr

How do I? (Keywords) Q- Search (36 +0)

mic29004

- Header Files
- Important Files
- Linker Files
- Source Files
- Libraries
- Loadables

asm1.asm

```
1 ; Replace with your application code
2
3 start:
4   inc r16
5   rjmp start
6
7
```

mic29004 - Dashboard

- Checksum: NA
- CRC32: 0x541EE624
- Packs
- ATmega_DFP (2.2.108)
- Compiler Toolchain
- AVRASM (v2.2.7) / Applications/microc
- Production Image: Optimization:
- Device support information: ATmega_f
- Memory
- Data 2,048 (0x800) bytes
- Data Used: 0 (0x0) Free: 2,048 (0x800) bytes
- Program 32,768 (0x8000) bytes
- Program Used: 36 (0x24) Free: 32,732 (0x7FCC) bytes
- Debug Tool
- Simulator
- Click for Simulated Peripherals
- Debug Resources
- Program BP Used: 0 Free: 1000
- Data BP Used: 0 Free: 1000

Output

SRAM Data Memory

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	SRAM Memory
110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
130	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
150	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
160	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Memory I/O Memor... Format Hex

I/O Memory (SRs)

Address	Name	Hex	Decimal	Bin
000	PC	0x0000	0	0000
001	R0	0x0000	0	0000
002	R1	0x0000	0	0000
003	R2	0x0000	0	0000
004	R3	0x0000	0	0000
005	R4	0x0000	0	0000
006	R5	0x0000	0	0000
007	R6	0x0000	0	0000
008	R7	0x0000	0	0000
009	R8	0x0000	0	0000
00A	R9	0x0000	0	0000
00B	R10	0x0000	0	0000
00C	R11	0x0000	0	0000
00D	R12	0x0000	0	0000
00E	R13	0x0000	0	0000
00F	R14	0x0000	0	0000
010	R15	0x0000	0	0000
011	R16	0x0000	0	0000
012	R17	0x0000	0	0000
013	R18	0x0000	0	0000
014	R19	0x0000	0	0000
015	R20	0x0000	0	0000
016	R21	0x0000	0	0000
017	R22	0x0000	0	0000
018	R23	0x0000	0	0000
019	R24	0x0000	0	0000
01A	R25	0x0000	0	0000
01B	R26	0x0000	0	0000
01C	R27	0x0000	0	0000
01D	R28	0x0000	0	0000
01E	R29	0x0000	0	0000
01F	R30	0x0000	0	0000
020	R31	0x0000	0	0000
021	PINB	0x0000	0	0000
022	DORB	0x0000	0	0000
023	PORTB	0x0000	0	0000
024	PINC	0x0000	0	0000
025	DORC	0x0000	0	0000
026	PORTC	0x0000	0	0000
027	PIND	0x0000	0	0000
028	DORC	0x0000	0	0000
029	PORTD	0x0000	0	0000
02A	PORTE	0x0000	0	0000
02B	PORTF	0x0000	0	0000
02C	TIFR0	0x0000	0	0000
02D	TIFR1	0x0000	0	0000
02E	TIFR2	0x0000	0	0000
02F	PCIFR	0x0000	0	0000
030	EIFR	0x0000	0	0000
031	EIFR	0x0000	0	0000
032	EIFR	0x0000	0	0000
033	EIFR	0x0000	0	0000
034	EIFR	0x0000	0	0000
035	EIFR	0x0000	0	0000
036	EIFR	0x0000	0	0000
037	EIFR	0x0000	0	0000
038	EIFR	0x0000	0	0000
039	EIFR	0x0000	0	0000
03A	EIFR	0x0000	0	0000
03B	EIFR	0x0000	0	0000
03C	EIFR	0x0000	0	0000
03D	EIFR	0x0000	0	0000
03E	EIFR	0x0000	0	0000
03F	EIFR	0x0000	0	0000
040	EIFR	0x0000	0	0000

Program Memory

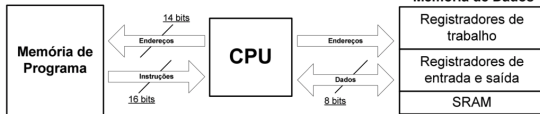
Line	Address	Opcodes	Label	Disassembly
1	0000	RPOP	init	LDI R16, 0x0F
2	0001	RPOP		OUT 0x04, R1
3	0002	RPOP	main	SBI 0x05, 5
4	0003	RPOP		LDI R19, 0x05
5	0004	RPOP		RCALL delay
6	0005	RPOP		CHI 0x05, 5
7	0006	RPOP		RCALL delay
8	0007	RPOP		RCALL delay
9	0008	RPOP		RJMP main
10	0009	RPOP	delay	SBR R17, R17
11	000A	RPOP		SBR R18, R18
12	000B	RPOP	l_delay	DEC R17
13	000C	RPOP		RNME l_delay
14	000D	RPOP		DEC R18
15	000E	RPOP		RNME l_delay
16	000F	RPOP		DEC R19
17	0010	RPOP		RNME l_delay
18	0011	RPOP		RET
19	0012	RPOP		FFFF
20	0013	RPOP		FFFF
21	0014	RPOP		FFFF
22	0015	RPOP		FFFF
23	0016	RPOP		FFFF
24	0017	RPOP		FFFF
25	0018	RPOP		FFFF
26	0019	RPOP		FFFF
27	001A	RPOP		FFFF
28	001B	RPOP		FFFF
29	001C	RPOP		FFFF
30	001D	RPOP		FFFF
31	001E	RPOP		FFFF
32	001F	RPOP		FFFF
33	0020	RPOP		FFFF
34	0021	RPOP		FFFF
35	0022	RPOP		FFFF
36	0023	RPOP		FFFF
37	0024	RPOP		FFFF
38	0025	RPOP		FFFF
39	0026	RPOP		FFFF
40	0027	RPOP		FFFF
41	0028	RPOP		FFFF
42	0029	RPOP		FFFF
43	002A	RPOP		FFFF
44	002B	RPOP		FFFF
45	002C	RPOP		FFFF
46	002D	RPOP		FFFF
47	002E	RPOP		FFFF
48	002F	RPOP		FFFF
49	0030	RPOP		FFFF
50	0031	RPOP		FFFF
51	0032	RPOP		FFFF

Memory Program M... Format Code

1:1 INS

Arquitetura vs. simulador

Microcontrolador ATmega328

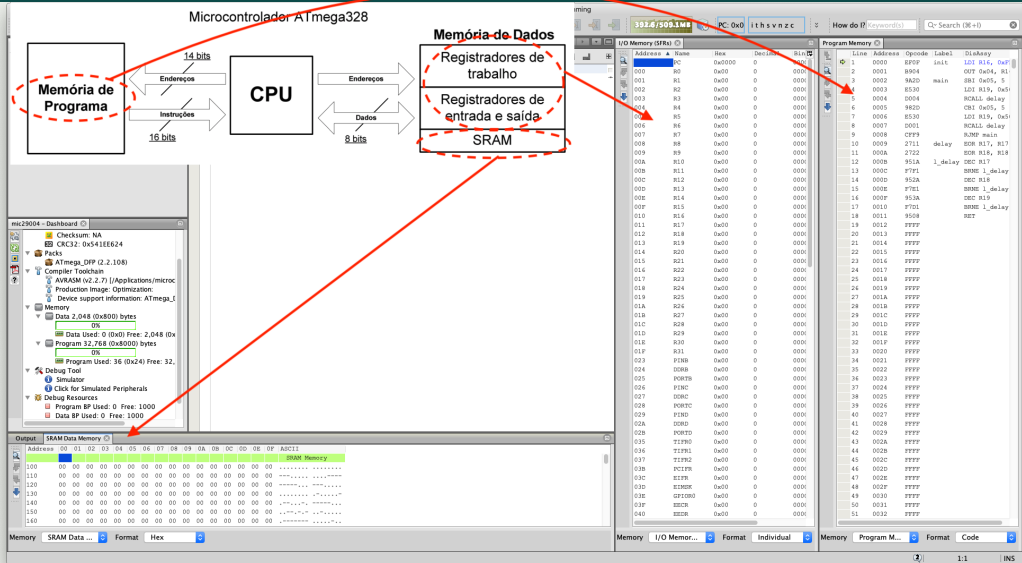


This screenshot shows the AVR Studio IDE interface. The **Project Explorer** on the left displays the project structure, including the **ATmega_DFP (2.2.108)** device and the **AVRASM (v2.2.7)** compiler. The **Output** window at the bottom shows the **SRAM Data Memory** window, which displays the memory contents in hexadecimal and ASCII. The **SRAM Data Memory** window shows the memory address range from 0x00 to 0x0F, with the data being 0x00 for all addresses.

This screenshot shows the AVR Studio IDE interface with the **I/O Memory** and **Program Memory** windows. The **I/O Memory** window displays the memory contents in hexadecimal and decimal, showing a range of addresses from 0x0000 to 0x000F. The **Program Memory** window displays the memory contents in hexadecimal and decimal, showing a range of addresses from 0x0000 to 0x000F. The **Program Memory** window also shows the **Disassembly** column, which contains assembly instructions such as **LDI R16, 0x00**, **OUT 0x04, R16**, and **main**.



Arquitetura vs. simulador



- Monte o programa:
 - *Debug -> Discrete Debugger Operation -> Build for Debugging*
- Inicie a simulação:
 - *Debug -> Discrete Debugger Operation -> Launch Debugger*
- Use os controles do debugger para a simular o código:



Fim!